

Sintassi di Base

Sintassi di Base

Le regole fondamentali della sintassi del C++.

Le regole di base del C++

Ogni linguaggio ha le sue regole grammaticali. In italiano mettiamo il punto alla fine della frase. In C++ dobbiamo mettere il **punto e virgola** alla fine di ogni istruzione. Se non rispetti queste regole, il programma non funziona.

Imparare la sintassi di base è come imparare l'alfabeto: prima lo conosci, poi puoi scrivere qualsiasi cosa.

La regola più importante: il punto e virgola

Ogni istruzione in C++ deve terminare con un **punto e virgola** (`;`). Pensa a esso come al punto fermo di una frase.

```
int x = 10;           // istruzione 1 – termina con ;
cout << "Ciao!";      // istruzione 2 – termina con ;
x = x + 1;           // istruzione 3 – termina con ;
```

Dimenticare il punto e virgola è l'errore più comune per chi inizia. Il compilatore te lo segnala subito.

Le parentesi graffe: i blocchi

Le parentesi graffe `{` e `}` delimitano un **blocco di codice**, cioè un gruppo di istruzioni che stanno insieme.

Pensa a un blocco come a un paragrafo in un testo: delimita un insieme di cose che vanno lette insieme.

```
int main() {  
    // tutto qui dentro è il "blocco" del main  
    int x = 5;  
    cout << x;  
}
```

Ogni `{` apre un blocco, ogni `}` lo chiude. Devono sempre essere in coppia.

Maiuscole e minuscole: fanno differenza

Il C++ distingue tra lettere maiuscole e minuscole. `cout`, `Cout` e `COUT` sono tre cose completamente diverse (solo `cout` è quello giusto).

```
int eta = 16;  
int Eta = 20;    // questa è un'altra variabile, diversa dalla prima!  
  
cout << eta;     // stampa 16  
cout << Eta;     // stampa 20
```

Regola pratica: scrivi tutto in minuscolo, tranne dove le convenzioni prevedono il contrario.

L'indentazione: non obbligatoria, ma fondamentale

In C++, gli spazi a inizio riga (l'indentazione) non cambiano il comportamento del programma. Questo funziona:

```
int main(){int x=5;cout<<x;return 0;}
```

Ma è illeggibile. Scrivi sempre il codice così:

```
int main() {
    int x = 5;
    if (x > 0) {
        cout << "positivo";
    }
    return 0;
}
```

Usa 4 spazi (o un tab) per ogni livello di indentazione. Il tuo futuro io te ne sarà grato.

I nomi delle variabili (identificatori)

Un **identificatore** è il nome che dai a una variabile, a una funzione, ecc. Pensa a esso come a un'etichetta su una scatola.

Regole per i nomi:

- Deve iniziare con una lettera (`a-z` , `A-Z`) o un underscore `_`
- Può contenere lettere, numeri e underscore
- Non può contenere spazi o simboli speciali (`@` , `!` , `-` , ecc.)
- Non può essere una parola riservata del linguaggio (come `int` , `if` , `return`)

```
// Nomi validi
int eta;
double altezza_media;
string nomeUtente;
int x1;

// Nomi non validi
int 2variabile;    // inizia con un numero - ERRORE
int nome utente;  // contiene uno spazio - ERRORE
int int;           // è una parola riservata - ERRORE
```

Le parole riservate

Le **parole riservate** sono parole che hanno già un significato speciale in C++. Non puoi usarle come nomi di variabili:

```
int, float, double, char, bool, void  
if, else, while, for, do, switch, case, break, continue  
return, class, struct, public, private, new, delete
```

Le convenzioni di stile

In C++ esistono tre modi principali per scrivere i nomi:

camelCase — la prima parola è minuscola, le successive iniziano con la maiuscola. Usato per le variabili:

```
int numeroStudenti;  
string nomeUtente;
```

snake_case — tutto minuscolo, parole separate da underscore. Anch'esso usato per le variabili:

```
int numero_studenti;  
string nome_utente;
```

PascalCase — ogni parola inizia con la maiuscola. Usato per i nomi delle classi:

```
class NomeClasse;  
struct DatiStudente;
```

Scegli uno stile e usalo in modo coerente in tutto il progetto.

I tipi di istruzione

In C++ ci sono diversi tipi di istruzioni. Ecco le principali:

```
// Dichiarazione di variabile – riserva uno spazio in memoria
int x;

// Assegnazione – mette un valore nella variabile
x = 10;

// Dichiarazione con inizializzazione – le due cose insieme
int y = 20;

// Chiamata a funzione – esegue un'azione
cout << "Ciao";

// Istruzione condizionale – prende una decisione
if (x > 0) {
    cout << "positivo";
}

// Istruzione di ritorno – termina la funzione
return 0;
```

Esempio completo

```
#include <iostream>
using namespace std;

int main() {
    // Dichiariamo e inizializziamo tre variabili
    int numero = 42;
    double pi = 3.14159;
    string messaggio = "Il numero magico è: ";

    // Stampiamo il messaggio e il valore di numero
    cout << messaggio << numero << endl;

    // Stampiamo il valore di pi
    cout << "Pi greco: " << pi << endl;

    // Calcoliamo il doppio di numero e lo stampiamo
    int doppio = numero * 2;
    cout << "Il doppio è: " << doppio << endl;

    return 0;
}
```

Output:

```
Il numero magico è: 42
Pi greco: 3.14159
Il doppio è: 84
```